

Evaluating Cross-Language Transfer for Refactoring Detection with Large Language Models

Siyuan Liu, Nabhan Suwanachote, Yutaro Kashiwa, Brittany Reid, Hajimu Iida
Nara Institute of Science and Technology, Nara, Japan

Abstract—The ability to detect refactoring is crucial to understanding software evolution. Many studies have proposed automated refactoring detection tools, such as RefactoringMiner and PyRef, but most are language-specific. Developing tools for new languages requires creating custom rules for each language, which is time-consuming and labor-intensive. This paper empirically evaluates the potential of Cross-Language Transfer (CLT) Learning using Large Language Models (LLMs) as a more general and cost-effective alternative for refactoring detection. Specifically, we compare CLT prompting against zero-shot prompting and native few-shot prompting. In CLT prompting, examples from one language (Java) are used to detect refactorings in another language (Python). In native few-shot prompting, examples are written in the same language as the target. Our results show that cross-language (Java→Python) transfer is highly effective, significantly outperforming the zero-shot prompting (Avg. F1 0.501 vs. 0.430). Surprisingly, it nearly matches the performance of native few-shot prompting (Python→Python, Avg. F1 0.506). These findings suggest that CLT is a promising direction for bypassing language-specific tool development.

Index Terms—Refactoring Detection, LLMs, Prompts

I. INTRODUCTION

Refactoring plays an important role in improving software quality. By definition, refactoring improves the internal code structure while preserving external behavior [1]. To study refactoring activities, many researchers have proposed approaches for detecting refactoring operations. Tsantalis *et al.* [2], [3], [4] developed RefactoringMiner for Java programs, and Atwi *et al.* [5] created PyRef for Python programs; these tools represent the current state-of-the-art in their respective languages. Both rely on complex, language-specific static rules that enable high detection accuracy. Through carefully crafted rules, these tools can detect various refactoring operations within individual languages.

However, this “per-language” development approach has significant limitations. Each new language requires substantial engineering effort to develop language-specific rules and parsers, creating a bottleneck for expanding refactoring detection. Previous studies have attempted to address this challenge. For example, Dilhara *et al.* [6] proposed detecting refactorings in Python by translating Python to Java to leverage RefactoringMiner. However, this approach proved fragile, as its reliance on syntactic translation failed to bridge the semantic divergence between Python’s dynamic typing and Java’s static paradigm, often introducing critical conversion errors [5].

More recently, data-driven approaches have shown promise. Afkhami *et al.* [7] and Li *et al.* [8] demonstrated that supervised Deep Learning (DL) models can effectively detect

refactorings across languages. However, these approaches typically require training models to learn refactoring patterns and language representations simultaneously from scratch or limited pre-training. In contrast, Large Language Models (LLMs) fundamentally differ by possessing extensive intrinsic multilingual knowledge acquired during massive pre-training. This enables the direct application of pre-aligned semantic knowledge to new languages, avoiding the construction of specialized model architectures.

In this context, Simoes *et al.* [9] employed LLMs with zero-shot prompting to classify refactoring types, concluding that LLM-based approaches are competitive with traditional rule-based tools and even clearly outperform them in some real-world cases. These findings suggest that foundation models can be extended to support multiple programming languages with minimal language-specific effort.

Building on this potential, we evaluated the performance of Cross-Language Transfer learning (CLT) for refactoring detection. CLT transfers learned patterns and structures from one programming language to another by exploiting commonalities in refactoring concepts across languages. This approach enables models to apply knowledge from high-resource languages (e.g., Java, characterized by abundant labeled datasets and mature analysis tools) to perform detection tasks in lower-resource languages (e.g., Rust, Python) [10], lowering the barrier to entry for robust, multi-language refactoring detection without requiring translation processes between languages.

In this study, we compare four prompting approaches to determine whether CLT effectively detects refactoring operations. In other words, we investigate whether LLMs learn abstract, language-agnostic concepts or rely on surface syntax [11]. We evaluate the performance of detecting refactorings in Python code changes using four prompting approaches: one with no examples, two with Java examples, and one with Python examples (representing the “performance ceiling” [12]). We assessed these on 161 code changes categorized into one of five refactoring types or non-refactoring.

Our results show that cross-language approaches outperform zero-shot prompting. Furthermore, cross-language prompt approaches demonstrate comparable performance with the native prompt approach (*i.e.*, using Python refactoring examples). These findings provide empirical motivation for future work to shift to more robust Cross-Language Fine-Tuning.

Replication Package: The data and programs used in our work are publicly available in the replication package [13].

II. PROMPTING TECHNIQUES

We introduce four distinct prompting approaches classified into two types of strategies. The classification is based on whether they use examples written in a high-resource language (e.g., Java) to detect refactoring operations in code written in a low-resource language (e.g., Python).

A. Native-Language prompting (“Monolingual” Prompts)

Native-language prompts use no examples or examples written in the same language as the code being analyzed.

Zero-shot prompting: Zero-shot provides only the task instruction without any examples. Figure 1 shows an example of zero-shot prompting. The instruction includes the task to analyze the provided code and determine whether a refactoring type is present, as well as the required output format (i.e., True or False). To obtain the classification, we provide the LLM with a query containing the low-resource language code to be analyzed (e.g., Python). The LLM then answers whether the specified refactoring type is included in the code. This prompt serves as a lower-bound baseline since it provides minimal information to the LLM compared with the other approaches.

(Instruction) Analyze the user’s code change and determine whether it constitutes a refactoring of Extract Method. Answer with “true” or “false”. (Query) Determine whether the following code change includes an Extract Method refactoring. Original Code: <code><Code before change></code> Modified Code: <code><Code after change></code>
--

Fig. 1: Example of Zero-shot prompting

Native Few-shot prompting: Native few-shot provides instructions along with examples of the refactoring types to be detected. These examples are “oracle” examples written in the target low-resource language. Figure 2 shows an example of native few-shot prompting. It uses the same template as the zero-shot prompt but includes examples written in Python, specifically, one positive example (i.e., code before/after refactoring changes) and one negative example (code before/after non-target (NT) refactoring changes, which includes other refactoring types or non-refactorings). However, this prompting approach is not practical because preparing these examples requires the same manual effort as rule-based approaches, which face the same challenge of needing language-specific examples. Therefore, we use this prompting as the performance upper bound [12] for this study.

(Instruction) Analyze the user’s code change and determine whether it constitutes a refactoring of Extract Method. Answer with “true” or “false”. Example 1 (Positive, Provided in Python) Original Code: <code><Code before refactoring change></code> Modified Code: <code><Code after refactoring change></code> Answer: <i>True</i> Example 2 (Negative, Provided in Python) Original Code: <code><Code before NT refactoring change></code> Modified Code: <code><Code after NT refactoring change></code> Answer: <i>False</i>
--

Fig. 2: Example of Native Few-shot prompting¹

B. Cross-Language prompting (“Cross-lingual” Prompts)

Cross-language prompts use examples written in a different language from the code being analyzed. In this study, the examples are in Java while the test cases are in Python.

Cross-language Few-shot prompting: This prompting provides the same instruction as native few-shot with examples written in a high-resource language as a hint. Figure 3 shows an example with the same structure, except that both the positive and negative examples are provided in Java code instead of Python. These are semantically equivalent translations of the Python examples, ensuring that language is the only variable.

(Instruction) Analyze the user’s code change and determine whether it constitutes a refactoring of Extract Method. Answer with “true” or “false”. Example 1 (Positive, Provided in Java) Original Code: <code><Code before refactoring change></code> Modified Code: <code><Code after refactoring change></code> Answer: <i>True</i> Example 2 (Negative, Provided in Java) Original Code: <code><Code before NT refactoring change></code> Modified Code: <code><Code after NT refactoring change></code> Answer: <i>False</i>
--

Fig. 3: Example of Cross-language Few-shot prompting¹

Cross-language Few-shot w/ descriptions prompting: Similarly, this prompting provides the same instruction as cross-language few-shot but adds natural language descriptions about what each refactoring type is, as shown in Figure 4. This strategy tests whether providing these explicit, conceptual descriptions can improve transfer performance. The descriptions for each refactoring type are available on online appendix [13].

(Instruction) Analyze the user’s code change and determine whether it constitutes a refactoring of Extract Method. Answer with “true” or “false”. (Description) The characteristics of Extract Method refactoring are as follows: This refactoring occurs when a segment of code is moved from an existing method (the original_method) and placed into a new, separate method (the new_method). The original_method is then modified to include a function call to this new_method in place of the removed code block. Example 1 (Positive, Provided in Java) Original Code: <code><Code before refactoring change></code> Modified Code: <code><Code after refactoring change></code> Answer: <i>True</i> Example 2 (Negative, Provided in Java) Original Code: <code><Code before NT refactoring change></code> Modified Code: <code><Code after NT refactoring change></code> Answer: <i>False</i>
--

Fig. 4: Example of Cross-language Few-shot w/ descriptions¹

III. EXPERIMENTAL SETUP

A. Research Questions

To investigate how Cross-Language Transfer Learning supports cross-language refactoring detection, we formulate the following research questions (RQs).

¹Queries are omitted due to space limitations but are identical to Zero-shot prompting (Figure 1)

RQ1: Does providing few-shot examples written in other languages improve the refactoring detection performance?

The “one-time annotation, multi-language use” paradigm offers immense economic value. While transfer techniques are highly successful in natural language processing [14], [15], [16], whether they translate to code refactoring remains an empirical question, particularly given the syntactic differences between languages like Java and Python. This RQ evaluates whether providing code examples written in different languages outperforms the Zero-shot baseline. (*i.e.*, Zero-shot vs CL Few-shot).

RQ2: How far is the performance gap between cross-language prompting and native few-shot prompting?

While cross-language examples may improve over Zero-shot, their practical value depends on how close they come to the native few-shot upper bound (*i.e.*, Native Few-shot vs Cross-Language Few-shot). A small gap would demonstrate that models can learn language-agnostic refactoring concepts, validating cross-language transfer as a cost-effective strategy. Conversely, a large gap would suggest that syntactic differences between languages limit transfer effectiveness. This performance gap also provides insight into the broader limitations of in-context learning, as a minimal gap would indicate that even native examples approach a low performance ceiling.

RQ3: Does providing refactoring descriptions improve cross-language prompt transfer performance?

If cross-language examples prove effective in RQ1, we investigate whether their performance can be enhanced with minimal effort. Natural language descriptions offer a low-cost way to provide abstract, language-agnostic guidance that may help models bridge syntactic gaps. Recent work shows that explicit natural language instructions can guide LLMs in complex code tasks [17]. We confirm whether adding refactoring descriptions to cross-language examples improves performance, potentially closing the gap observed in RQ3.

B. Task and Dataset

Rather than asking the LLM to select from multiple refactoring types simultaneously, we decompose the problem into five independent binary classification tasks (*e.g.*, “Is this an Extract Method refactoring? true/false”). This simplifies the task and ensures consistent output formatting across all queries.

To conduct our evaluation, we employ GPT-4.1-mini, a representative high-performance LLM widely adopted in recent research [18], [19]. All experiments use a temperature of 0.0 to ensure deterministic reproducibility.

Our evaluation uses a dataset from a previous study [20], which includes 1,914 refactoring examples spanning 15 types of refactoring operations from 136 real-world open-source projects. We focus on method and variable-level refactoring types: EXTRACT METHOD, INLINE METHOD, RENAME VARIABLE, EXTRACT VARIABLE, and INLINE VARIABLE. The resulting dataset contains 161 refactoring operations, distributed approximately evenly across six categories: the five refactoring types plus a “non-refactoring” category. Non-refactoring

changes are those that alter program behavior, such as functional enhancements or bug fixes, as opposed to behavior-preserving refactorings. These non-refactoring examples were included in the original dataset as false positives for refactoring detection.

C. Performance Metrics

Precision and Recall: These metrics, which are crucial for diagnosing model behavior, are reported to dissect and explain key performance anomalies. Specifically, Precision quantifies the rate of false positives (FP), revealing whether the model is labeling non-refactoring changes too broadly. Recall, on the contrary, quantifies the rate of false negatives (FN), indicating how often the model misses true refactoring instances.

F1-score: Given the inherent class imbalance of our binary classification tasks (a few positive instances vs. many negative ones), the F1-score is our primary metric for overall performance. It is used for high-level comparisons in our main results tables. We use F1-score as a primary metric.

IV. RESULTS

This section presents our empirical results, organized by research question. Table I summarizes the classification performance. Due to space constraints, we report only F1 scores; precision and recall are available in our replication package [13].

RQ1: Performance of Cross-Language Transfer (Zero-shot vs CL Few-shot)

With the Zero-shot prompting, the average F1-score is 0.430. When we add few-shot examples from Java (*i.e.*, CL Few-shot), the average F1-score rises to 0.501. This represents a 16.5% relative improvement. CL Few-shot outperformed Zero-shot across most refactoring types except for INLINE VARIABLE.

However, surprisingly, Zero-shot achieved a higher F1-score than CL Few-shot (even Native Few-shot) for detecting INLINE VARIABLE. This may be because the few-shot examples were likely insufficient to capture the precise structural complexity, causing the model to over-constrain its predictions compared to its inherent capabilities. The overall improvements suggest that few-shot learning provides meaningful benefits over the zero-shot approach.

Answer to RQ1: Providing examples in a source language (*i.e.*, Java) appears to improve detection performance in the target language (*i.e.*, Python).

RQ2: Performance Limitation (Native vs CL Few-shot)

Contrary to our expectations, the performance gap between CL Few-shot and Native Few-shot was small. CL Few-shot achieved 0.501, which is 99% of Native Few-shot’s 0.506. The gap was minimal for most refactoring types: RENAME VARIABLE (0.546 vs. 0.552) and EXTRACT VARIABLE (0.429 vs. 0.452). EXTRACT METHOD was the exception, where Native Few-shot (0.758) outperformed CL Few-shot (0.697) by

TABLE I: F1-Scores for refactoring detection by prompt strategy using GPT-4.1-mini. **Bold** values indicate the highest F1 score for each refactoring type.

Refactoring Type	Zero-shot ($\rightarrow$ Python)	CL Few-shot (Java $\rightarrow$ Python)	CL Few-shot with desc (Java $\rightarrow$ Python)	Native Few-shot (Python $\rightarrow$ Python)
Used in RQ1	○	○		
Used in RQ2		○		○
Used in RQ3		○	○	○
EXTRACT METHOD	0.640	0.697	0.754	0.758
INLINE METHOD	0.459	0.583	0.619	0.520
RENAME VARIABLE	0.490	0.546	0.542	0.552
EXTRACT VARIABLE	0.312	0.429	0.393	0.452
INLINE VARIABLE	0.250	0.125	0.125	0.125
Average	0.430	0.501	0.512	0.506

a larger margin. The modest Native Few-shot performance (0.506) indicates a relatively low ceiling for this task.

Answer to RQ2: The performance gap between CL Few-shot and Native Few-shot is not large, implying that Cross-Language Transfer can be an effective and cost-efficient strategy.

RQ3: Performance of Refactoring Description (CL Few-shot with desc v.s. CL Few-shot without desc)

CL Few-shot with desc achieved an average F1-score of 0.512, a modest 2.2% improvement over CL Few-shot (0.501). With this gain, CL Few-shot with description now outperforms Native Few-shot.

However, the improvement was inconsistent across refactoring types. CL Few-shot with desc improved on EXTRACT METHOD (0.754 vs. 0.697) and INLINE METHOD (0.619 vs. 0.583), but performed worse on RENAME VARIABLE (0.542 vs. 0.546) and EXTRACT VARIABLE (0.393 vs. 0.429).

Answer to RQ3: Providing detailed natural language descriptions about refactoring types does not yield a reliable or consistent performance benefit.

V. LESSON LEARNED

In this section, we interpret the findings presented in Section IV, weaving together the performance data and key anomalies to build our core argument. We discuss the viability of Cross-Language Transfer (CLT) and the limitations of In-Context Learning (*e.g.*, the native fewshot prompting).

A. The Viability and Potential of CLT (RQ1)

RQ1 confirms that the CL Few-shot (Java $\rightarrow$ Python) achieved an 16.5% relative performance increase over the Zero-shot baseline. This indicates that cross-language few-shot prompting is a highly viable strategy.

The EXTRACT METHOD case illustrates this pattern clearly. Zero-shot achieved high recall (0.828) but very low precision (0.522). This suggests the model was “over-guessing” extract method refactorings. Introducing Java examples (*i.e.*, CL Few-shot) increased precision significantly by (19.1%) to 0.622, while recall dropped only slightly to 0.793.

This shift demonstrates that the model is not just guessing, but learning a more precise decision boundary from the Java examples. This supports our claim that the LLM is indeed capable of learning and transferring the abstract refactoring patterns encoded in the source language (Java) examples to the target language (Python).

The INLINE METHOD case, where CL Few-shot (0.583) outperformed Native Few-shot (0.520), offers deeper, noteworthy evidence for the potential of CLT. This counter-intuitive F1 score reversal is driven by CL Few-shot achieving both higher precision (0.539 vs. 0.464) and higher recall (0.636 vs. 0.591) than Native Few-shot.

We hypothesize that in this case, the Java examples, with their stricter syntax, provided clearer and less ambiguous patterns than their Python equivalents. The model successfully captured these patterns, improving both precision and recall despite the language barrier. This suggests that CLT should not be viewed solely as a compromise solution. In certain scenarios, cross-language examples may provide signals that are comparable to, or potentially exceed, those from native examples.

B. The Limitations and Brittleness of the In-Context Learning Paradigm (RQ2 & RQ3)

In-Context Learning (ICL), formalized by Brown *et al.* [21], is a paradigm where LLM learns to perform a task at inference time without updating its parameters. The model is conditioned on a natural language prompt that includes a task description and a set of input-output demonstrations, allowing it to recognize and replicate the desired pattern for a new query. The native few-shot prompting approach is categorized into ICL. In the following, we discuss how our findings reveal fundamental limitations of ICL’s paradigm for refactoring detection.

In answering RQ3, the average performance gap between CL Few-shot (0.501) and Native Few-shot (0.506) proved unexpectedly small. However, this should not be interpreted as evidence that CLT has achieved near-perfect transfer efficiency. Rather, this reveals a deeper issue: the observed performance ceiling of the ICL paradigm itself appears limited in our experiment. This observation aligns with recent studies showing that ICL “falls short” on

“specification-heavy” tasks [22]. An average F1-score of 0.506 for Native Few-shot remains relatively low for production use. This indicates that ICL may be insufficient to address the full complexity of refactoring detection.

This low ceiling appears to be linked to the inherent brittleness of the ICL paradigm. Our data provides two key cases to support this view. The first case involves `INLINE VARIABLE`, which demonstrates a generalization failure. The “negative learning” phenomenon (Native Few-shot < Zero-shot) is driven by a catastrophic collapse in recall (from 0.368 down to 0.105), while precision also slightly decreased (0.189 vs. 0.154). This strongly suggests a failure in ICL’s generalization ability. When faced with complex, “impure” real-world samples, the “textbook” examples in Native Few-shot not only failed to provide a useful pattern but actively confused the model, causing it to miss the vast majority of true positives it would have otherwise found.

The second case involves `INLINE METHOD`, which demonstrates instability. As discussed in Section V-A, the phenomenon where CL Few-shot outperforms Native Few-shot, while revealing CLT’s potential, also exposes ICL’s high sensitivity to the specific examples chosen. This finding aligns with prior work [23], which documents that ICL’s performance is highly sensitive to the format, selection, and even the order of in-context examples. This unpredictability makes it an unreliable strategy in engineering practice.

The performance of CL Few-shot with desc (Avg. F1 0.512) further reinforces the limitations of In-Context Learning. The average benefit over CL Few-shot was marginal (answering RQ3), and its performance was highly inconsistent across categories (*e.g.*, performance decreased for `RENAME VARIABLE`). This demonstrates that attempting to breach the In-Context Learning’s bottleneck with more complex prompt engineering yields limited and unpredictable returns.

VI. FUTURE DIRECTION

This research successfully establishes the viability of Cross-Language Transfer (CLT) for refactoring detection, but it simultaneously highlights the limitations and inherent brittleness of the few-shot prompting knowledge transfer method. The future direction of this work should concentrate on overcoming these limitations through robust generalization methods and a significant expansion of the scope of evaluation. In the following, we outline two critical directions for future research.

More Languages and More Refactoring Types. The current study serves as an initial evaluation, focusing on a single language pair (Java→Python) and evaluating only five refactoring types, which are limited to method-level or variable-level refactoring. Future research must aim to expand this scope to validate the generality of CLT. This expansion should include testing more target languages, such as Rust, beyond Python, as the evaluated cross-language approach is applicable to various languages. Furthermore, for simplicity, the current study targeted only a limited set of refactoring types, so future efforts must address a wider range of refactoring types to confirm the full utility of this approach.

Fine-tuning LLMs for Cross-Language Transfer. Our results show that CLT can transfer knowledge across languages without preparing examples for each language. This provides strong empirical motivation for exploring Cross-Language Fine-Tuning as an alternative to prompting-based approaches.

Fine-tuning on a large source-language refactoring dataset may overcome the brittleness of few-shot prompting by embedding a more robust understanding of refactoring patterns. Prior work consistently shows that fine-tuning specializes models to complex domains more effectively than few-shot prompting [24], [25]. The model’s transfer capabilities could then apply this learned knowledge to target languages, potentially surpassing the low performance ceiling observed in our prompting experiments.

Our primary future work will empirically test this Cross-Language Fine-Tuning hypothesis. We plan to construct a large-scale, high-quality refactoring dataset from Java (source language) and fine-tune a base model on this dataset. We will then measure the model’s zero-shot transfer performance on our existing Python (target language) test set. This will enable direct comparison against the limitations of prompting approaches observed in this study. A subsequent goal will be to benchmark the fine-tuned model against existing refactoring detection tools. We hypothesize that Cross-Language Fine-Tuning will demonstrate more stable performance and significantly exceed the low performance ceiling (Avg. F1 0.506) that limited prompting-based approaches.

VII. CONCLUSION

This paper investigated Cross-Language Transfer (CLT) learning, using few-shot Java examples to detect refactorings in Python, bypassing the need for language-specific tools. Our findings confirm CLT is robust and viable. The Cross-Language Few-shot (Java→Python) approach significantly outperformed the Zero-shot baseline (Avg. F1 0.501 vs. 0.430), demonstrating that LLMs can learn and transfer language-agnostic refactoring patterns. However, CLT performance nearly matched Native Few-shot (0.501 vs. 0.506). This minimal gap, combined with modest Native F1-scores, reveals a critical limitation: few-shot prompting is an insufficient and brittle knowledge transfer method. The low Native Few-shot performance (0.506) indicates a low ceiling for this approach. This brittleness appears through instability (CLT sometimes outperformed Native Few-shot for Inline Method) and generalization failure (Inline Variable’s recall collapse). Efforts to improve CLT using detailed refactoring descriptions provided only marginal, inconsistent benefits (Avg. F1 0.512).

ACKNOWLEDGMENT

We gratefully acknowledge the financial support of JSPS KAKENHI grants (JP24K02921, JP25K21359), as well as JST PRESTO grant (JPMJPR22P3), ASPIRE grant (JPMJAP2415), and AIP Accelerated Program (JPMJCR25U7).

REFERENCES

- [1] M. Fowler, *Refactoring - Improving the Design of Existing Code*, ser. Addison Wesley object technology series, 1999.
- [2] N. Tsantalis, A. Ketkar, and D. Dig, "RefactoringMiner 2.0," *IEEE Trans. Software Eng.*, vol. 48, no. 3, pp. 930–950, 2022.
- [3] P. Alikhanifard and N. Tsantalis, "A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 40:1–40:63, 2025.
- [4] N. Tsantalis, M. Mansouri, L. M. Eshkevari, D. Mazinanian, and D. Dig, "Accurate and efficient refactoring detection in commit history," in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, 2018, pp. 483–494.
- [5] H. Atwi, B. Lin, N. Tsantalis, Y. Kashiwa, Y. Kamei, N. Ubayashi *et al.*, "PYREF: refactoring detection in Python projects," in *21st IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2021, pp. 136–141.
- [6] M. Dilhara, A. Ketkar, N. Sannidhi, and D. Dig, "Discovering repetitive code changes in Python ML systems," in *44th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2022, pp. 736–748.
- [7] M. M. Afkhami, I. H. Moghadam, V. Zaytsev, M. Ashoori, and H. Bazmandegan, "Refactoring detection across languages: Leveraging Java-trained models for detecting class-level refactorings in Kotlin," in *Software Engineering and Advanced Applications - 51st Euromicro Conference (SEAA)*, 2025, pp. 46–63.
- [8] T. Li and Y. Zhang, "Multilingual code refactoring detection based on deep learning," *Expert Syst. Appl.*, vol. 258, p. 125164, 2024.
- [9] P. Simões, R. Gheyi, R. Melo, J. Oliveira, M. Ribeiro, and W. K. G. Assunção, "RefModel: detecting refactorings using foundation models," in *Proceedings of the 39th Brazilian Symposium on Software Engineering (SBES)*, 2025, pp. 811–817.
- [10] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán *et al.*, "Unsupervised cross-lingual representation learning at scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 8440–8451.
- [11] S. Haroon, A. F. Khan, A. Humayun, W. Gill, A. H. Amjad, A. R. Butt *et al.*, "How accurately do large language models understand code?" *CoRR*, vol. abs/2504.04372, 2025.
- [12] J. Shin, C. Tang, T. Mohati, M. Nayeibi, S. Wang, and H. Hemmati, "Prompt engineering or fine-tuning: An empirical assessment of LLMs for code," in *22nd IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 2025, pp. 490–502.
- [13] S. Liu, N. Suwanachote, Y. Kashiwa, B. Reid, and H. Iida, "Replication Package for "Evaluating Cross-Language Transfer for Refactoring Detection with Large Language Models"," <https://github.com/ryushien-z/C LT4RD-replication-package>, 2025.
- [14] S. Wu and M. Dredze, "Beto, bentz, becas: The surprising cross-lingual effectiveness of BERT," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 833–844.
- [15] D. I. Adelani, G. Neubig, S. Ruder, S. Rijhwani, M. Beukman, C. Palen-Michel *et al.*, "MasakhaNER 2.0: africa-centric transfer learning for named entity recognition," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022, pp. 4488–4508.
- [16] A. Lauscher, V. Ravishankar, I. Vulic, and G. Glavas, "From zero to hero: On the limitations of zero-shot language transfer with multilingual transformers," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 4483–4499.
- [17] J. White, S. Hays, Q. Fu, J. Spencer-Smith, and D. C. Schmidt, "ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design," *CoRR*, vol. abs/2303.07839, 2023.
- [18] Y. Wang, L. Yang, Y. Tian, K. Shen, and M. Wang, "Co-evolving LLM coder and unit tester via reinforcement learning," *CoRR*, vol. abs/2506.03136, 2025.
- [19] Z. Wang, J. Zhu, B. Tang, Z. Li, F. Xiong, J. Yu *et al.*, "Jigsaw-r1: A study of rule-based visual reinforcement learning with jigsaw puzzles," *Trans. Mach. Learn. Res.*, vol. 2025, 2025.
- [20] S. Wang, X. Hu, X. Xia, and X. Wang, "ActRef: enhancing the understanding of python code refactoring with action-based analysis," *CoRR*, vol. abs/2505.06553, 2025.
- [21] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, "Language models are few-shot learners," in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901.
- [22] H. Peng, X. Wang, J. Chen, W. Li, Y. Qi, Z. Wang *et al.*, "When does in-context learning fall short and why? A study on specification-heavy tasks," *CoRR*, vol. abs/2311.08993, 2023.
- [23] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 12 697–12 706.
- [24] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal *et al.*, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Proceedings of the 36th International Conference on Neural Information Processing Systems (NeurIPS)*, 2022, pp. 1950–1965.
- [25] A. Edwards and J. Camacho-Collados, "Language models for text classification: Is in-context learning enough?" in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC/COLING)*, 2024, pp. 10 058–10 072.